

Technical Report

Enhancing and Expanding Blockchain, Federated Learning (FL), and Homomorphic Encryption (HE) Integration

Ibram Ghali
School of Information Technology
Deakin University Melbourne, Victoria, Australia

Contents

Abstract.....4

1. Introduction4

2. Analysis of the Current Work.....4

2.1 Strengths.....4

2.1.1 Privacy Protection4

2.1.2 Security via Blockchain.....5

2.2 Limitations6

2.2.1 Scalability Bottlenecks.....6

2.2.2 Computational Overhead6

2.2.3 Domain Narrowness6

2.2.4 Handling Non-IID Data6

2.2.5 Limited Testing in Real-World Environments.....6

2.2.6 Privacy vs. Utility Trade-Off7

2.2.7 Adversarial Robustness.....7

3. Proposed Enhancements7

3.1 Enhancing Scalability.....7

3.1.1 Sharded Blockchain Architecture7

3.1.2 Layer 2 Solutions7

3.1.3 Asynchronous Federated Learning.....8

Optimizing Computational Efficiency8

3.1.4 Hybrid Encryption Techniques8

3.1.5 Edge Computing Integration8

3.1.6 Compressed Updates9

3.2 Improving Security and Robustness.....9

3.2.1 Zero-Knowledge Proofs (ZKP).....9

3.2.2 Dynamic Smart Contracts9

3.2.3 Adversarial Robustness Testing10

3.3 Handling Non-IID Data10

3.3.1 Clustered Federated Learning10

3.3.2 Meta-Learning10

4. Technical Implementation Plan: Detailed Guide11

4.1 Framework Architecture.....11

4.1.1 Multi-Tier Blockchain System.....11

4.1.2 Dynamic Model Aggregation Rules.....11

4.2 Operational Workflow12

4.2.1	Data Preprocessing and Encryption	12
4.2.2	Blockchain-Integrated Federated Learning	12
4.2.3	Consensus and Aggregation	13
4.2.4	Global Model Distribution	13
4.3	Tools and Technologies	13
4.3.1	Blockchain Platforms	13
4.3.2	Federated Learning Frameworks	14
4.3.3	Encryption Libraries	14
4.3.4	Programming Frameworks	14
4.4	Deployment Plan	14
4.4.1	Testing Environment	14
4.4.2	Deployment Stages	14
4.4.3	Monitoring and Maintenance	14
4.5	Key Benefits of the Implementation Plan	15
6.	Conclusion	15
7.	References	16
	Acknowledgment	17

Abstract

This report critically evaluates the integration of Blockchain, Federated Learning (FL), and Homomorphic Encryption (HE) based on the previous works provided. It identifies areas for improvement and proposes advanced solutions to enhance privacy, scalability, efficiency, and applicability across diverse domains. The recommended improvements focus on addressing computational overhead, ensuring robust security, improving model accuracy, and expanding use cases beyond financial systems into healthcare, IoT, and smart cities. This comprehensive report aims to pave the way for deploying scalable and efficient decentralized learning systems in real-world environments.

1. Introduction

The integration of Blockchain, FL, and HE has demonstrated great potential in enabling privacy-preserving and secure collaborative learning across multiple domains. The previous works highlighted successful implementations in financial services, addressing challenges like data privacy, regulatory compliance, and security. However, scalability, computational efficiency, and the adaptability of these frameworks remain significant concerns.

This report builds on the strengths of the earlier works while addressing their limitations. It explores novel enhancements, domain-specific extensions, and operational optimizations to elevate the performance and impact of Blockchain-FL-HE systems.

2. Analysis of the Current Work

This section evaluates the existing integration of Blockchain, Federated Learning (FL), and Homomorphic Encryption (HE) as presented in previous works. The analysis identifies the strengths that establish the foundation for secure and privacy-preserving decentralized learning, as well as the limitations that hinder scalability, efficiency, and adaptability. By understanding the current state, this section lays the groundwork for the enhancements proposed in later sections.

2.1 Strengths

2.1.1 Privacy Protection

- Federated Learning (FL): FL ensures that raw data remains localized at individual nodes, addressing privacy concerns by avoiding centralized data

storage. This aligns with stringent data protection regulations such as GDPR and HIPAA.

Example: In financial fraud detection, banks can train models collaboratively without sharing sensitive transaction data.

- Homomorphic Encryption (HE): Enables computations on encrypted data, ensuring data privacy even during processing. Model updates are encrypted before being transmitted, protecting sensitive information even in transit.

Example: HE ensures that even if model updates are intercepted during transmission, they remain unreadable.

2.1.2 Security via Blockchain

- Immutable Ledger: Blockchain maintains a tamper-proof record of all transactions (e.g., model updates), ensuring transparency and trust.

Example: In SPL management, each institution's updates to the suspicious persons list are verifiable and immutable.

- Decentralization: Removes reliance on a central aggregator, eliminating single points of failure. Distributed consensus mechanisms like Proof of Stake (PoS) or Practical Byzantine Fault Tolerance (PBFT) validate updates, ensuring network integrity.

- Smart Contracts: Automate validation and aggregation processes, reducing the risk of human error or malicious manipulation.

Example: Smart contracts ensure only valid updates are aggregated into the global model.

- Collaborative Learning with Diverse Data Federated Learning enables the development of global models that leverage data from multiple sources, improving accuracy and robustness.

Example: In cross-border transaction monitoring, the global model benefits from learning fraud patterns across diverse financial institutions.

- Domain-Specific Success Financial applications such as credit card fraud detection, SPL management, and credit scoring demonstrate the feasibility of Blockchain-FL-HE integration.

These use cases highlight the value of combining privacy, security, and collaborative learning.

2.2 Limitations

2.2.1 Scalability Bottlenecks

Blockchain Latency: Consensus mechanisms such as PoS and PBFT struggle to handle high transaction volumes in networks with numerous participants. Latency increases as nodes validate transactions, limiting real-time applications.

Example: In cross-border payments, delays in transaction validation could impact fraud detection efficiency. **High Node Count:** As the number of nodes increases, the network faces bottlenecks in processing and storing transactions.

2.2.2 Computational Overhead

Homomorphic Encryption: HE adds significant processing demands due to its computationally intensive nature, especially for large-scale datasets or frequent model updates.

Example: Encrypting model updates in SPL management can delay real-time aggregation.

Smart Contracts: The execution of complex smart contracts for validation and aggregation increases computational load.

2.2.3 Domain Narrowness

The current frameworks focus heavily on financial applications, leaving other critical domains such as healthcare, IoT, and smart cities underexplored.

These domains have distinct privacy, scalability, and operational requirements that the current models do not address.

2.2.4 Handling Non-IID Data

Issue: Data across nodes in real-world scenarios often follow non-independent and identically distributed (non-IID) patterns, making it difficult for the global model to generalize effectively.

Impact: Non-IID data leads to slower model convergence and reduced performance.

Example: Fraud patterns vary significantly across financial institutions, creating a challenge for the global model to learn universally applicable patterns.

2.2.5 Limited Testing in Real-World Environments

Existing works rely on theoretical or simulated environments, which do not capture the complexity of real-world networks.

Real-world challenges such as adversarial attacks, network outages, or regulatory compliance intricacies remain untested.

2.2.6 Privacy vs. Utility Trade-Off

While HE ensures strong privacy, it limits the types of computations that can be performed efficiently.

Example: In fraud detection, encrypted updates reduce computational risks but may lead to less efficient learning.

2.2.7 Adversarial Robustness

Current models lack sufficient defenses against adversarial threats such as:

Model Poisoning: Malicious updates skew the global model.

Sybil Attacks: Attackers flood the network with fake nodes to manipulate consensus.

3. Proposed Enhancements

This section outlines the proposed solutions to address the limitations identified in the analysis of existing frameworks. The enhancements focus on improving scalability, computational efficiency, security, and adaptability for real-world applications. These changes aim to establish a robust, efficient, and privacy-preserving framework that integrates Blockchain, Federated Learning (FL), and Homomorphic Encryption (HE).

3.1 Enhancing Scalability

3.1.1 Sharded Blockchain Architecture

- Concept: Partition the blockchain into shards, each responsible for managing a subset of transactions or participants, enabling parallel processing.
- Implementation: Assign nodes to specific shards based on geographic regions, institutional types, or transaction categories. Synchronize shards through cross-shard communication protocols to ensure consistency.
- Example Use Case: In fraud detection, one shard processes updates from institutions in North America, while another shard handles those from Europe.
- Benefits:
 - Boosts transaction throughput by parallelizing operations.
 - Reduces latency and computational load on individual nodes.

.

3.1.2 Layer 2 Solutions

Concept: Use Layer 2 scaling solutions to handle off-chain computations while keeping the blockchain for final consensus and record-keeping.

Implementation:

Off-chain rollups aggregate and validate updates before submitting a single transaction to the blockchain.

State channels facilitate direct peer-to-peer interactions without involving the blockchain for every transaction.

Example Use Case: Batch multiple encrypted model updates into a single transaction for blockchain validation.

Benefits: Reduces congestion on the main chain And speeds up consensus and improves scalability.

3.1.3 Asynchronous Federated Learning

Concept: Allow nodes to submit model updates asynchronously, reducing synchronization delays.

Implementation: Design an aggregation protocol that incorporates updates as they arrive, rather than waiting for all nodes to complete training.

Example Use Case: In SPL management, institutions with varying computational capacities can submit updates independently.

Benefits: Improves efficiency in geographically distributed networks and reduces idle time for faster model iteration.

Optimizing Computational Efficiency

3.1.4 Hybrid Encryption Techniques

Concept: Combine Homomorphic Encryption (HE) with Differential Privacy (DP) to balance privacy and computational overhead.

Implementation: Encrypt only sensitive model parameters while adding DP noise to less critical features.

Example Use Case: Encrypt fraud detection model weights while adding noise to auxiliary features like time of transaction.

Benefits: Reduces encryption overhead while maintaining privacy and improves scalability for large-scale applications.

3.1.5 Edge Computing Integration

Concept: Perform data preprocessing and encryption tasks on edge devices before submitting updates to the network.

Implementation: IoT devices or institutional servers preprocess local datasets, apply HE, and submit encrypted updates.

Example Use Case: Smart meters in a smart city aggregate and encrypt energy usage data before contributing to a global optimization model.

Benefits: Reduces central server workloads. Improves real-time processing for latency-sensitive applications.

3.1.6 Compressed Updates

Concept: Reduce the size of model updates using gradient sparsification or quantization techniques.

Implementation: Retain only the most significant gradients during transmission. Quantize floating-point updates to lower-precision integers.

Example Use Case: Compress fraud detection model updates before submitting them to the blockchain.

Benefits: Lowers communication costs and bandwidth usage and speeds up the aggregation process.

3.2 Improving Security and Robustness

3.2.1 Zero-Knowledge Proofs (ZKP)

Concept: Validate model updates without revealing their contents.

Implementation: Nodes submit ZKPs to prove compliance with network rules (e.g., update integrity, compliance with format).

Example Use Case: Financial institutions prove the validity of encrypted SPL contributions without disclosing sensitive data.

Benefits: Enhances privacy without compromising validation and increases trust among participants.

3.2.2 Dynamic Smart Contracts

Concept: Adjust aggregation rules dynamically based on network activity and node reliability.

Implementation: Use smart contracts to assign weights to updates based on node reputation and data quality.

Example Use Case: Nodes with a history of high-quality updates have greater influence on the global model.

Benefits: Ensures the robustness of the global model and mitigates the impact of malicious or low-quality updates.

3.2.3 Adversarial Robustness Testing

Concept: Simulate and test the framework against potential attacks.

Implementation: Deploy anomaly detection systems to identify and reject malicious updates and Simulate Sybil attacks or model poisoning to evaluate defense mechanisms.

Example Use Case: Identify poisoned updates in a healthcare diagnostic model trained across multiple hospitals.

Benefits: Ensures system resilience against adversarial threats and builds trust in the integrity of the framework.

3.3 Handling Non-IID Data

3.3.1 Clustered Federated Learning

Concept: Group nodes with similar data distributions into clusters for localized training.

Implementation:

Train cluster-specific models before aggregating them into a global model.

Example Use Case: Banks in different regions train models for localized fraud patterns, which are then aggregated globally.

Benefits: Improves model accuracy and convergence rate and reduces the impact of heterogeneous data distributions.

3.3.2 Meta-Learning

Concept: Teach global models to adapt dynamically to local data distributions.

Implementation: Introduce meta-learning layers that allow the global model to adjust to new environments.

Example Use Case: Global healthcare models adapting to regional disease prevalence data.

Benefits: Enhances generalizability and reduces training iterations needed for convergence.

4. Technical Implementation Plan: Detailed Guide

The technical implementation plan is a blueprint for executing the proposed enhancements in integrating Blockchain, FL, and HE. It addresses the architectural setup, operational workflows, and necessary tools and technologies to ensure a scalable, secure, and efficient system.

4.1 Framework Architecture

4.1.1 Multi-Tier Blockchain System

Overview: The blockchain architecture is divided into two tiers to optimize performance, scalability, and compliance:

- **Tier 1 (Permissioned Blockchain):** Handles critical operations such as storing encrypted model updates, validating updates, and ensuring privacy-sensitive transactions.

Example Platform: Hyperledger Fabric, which supports permissioned networks with high transaction throughput.

Features: Role-based access control to restrict data visibility to authorized participants. Smart contracts for automated validation and rule enforcement.

- **Tier 2 (Public Blockchain):** Manages metadata, audit trails, and transparency for compliance and traceability.

Example Platform: Ethereum or Polygon for decentralized, publicly verifiable records.

Features: Stores cryptographic hashes of encrypted updates from Tier 1 for immutability. Facilitates transparency for regulators and external auditors.

Benefits: Offloads computationally intensive operations to the permissioned blockchain, ensuring high performance. Public blockchain provides tamper-proof evidence of compliance without exposing sensitive details.

4.1.2 Dynamic Model Aggregation Rules

Implementation:

- Smart contracts on Tier 1 dynamically adjust aggregation rules based on:
- Node reliability: Weighted contributions for nodes with consistent performance.
- Data quality: Prioritize updates from nodes with higher-quality datasets.

Aggregation algorithms include secure averaging (e.g., Federated Averaging) and gradient sparsification.

Benefits: Ensures the global model is robust and representative and reduces the impact of malicious or low-quality updates.

4.2 Operational Workflow

4.2.1 Data Preprocessing and Encryption

Edge Processing: Nodes (e.g., institutions, IoT devices) preprocess data locally, including normalization, encoding, and feature extraction. Sensitive data is encrypted using Homomorphic Encryption (HE).

Library: PyCryptodome or TenSEAL.

Optimization: Use selective encryption for high-sensitivity features to reduce computational overhead.

Implementation:

Encrypt model gradients or weights before sharing with the blockchain.

Benefits:

Ensures compliance with privacy regulations (e.g., GDPR, HIPAA).

Reduces the risk of data leakage during transmission.

4.2.2 Blockchain-Integrated Federated Learning

Local Model Training:

Each node trains a model on its local, encrypted data using machine learning frameworks like PyTorch or TensorFlow.

Post-training, the model updates (gradients or parameters) are encrypted.

Data Submission:

Encrypted updates are submitted to the blockchain via smart contracts.

Smart contracts validate:

Update integrity (e.g., non-malicious data).

Compliance with network rules.

Metadata (e.g., update hash, timestamp) is simultaneously recorded on the public blockchain for transparency.

Tools:

Web3.py: For smart contract interactions.

Hyperledger SDK: For node communication and transaction handling.

4.2.3 Consensus and Aggregation

Consensus Mechanism:

Tier 1 (Permissioned Blockchain):

Uses Practical Byzantine Fault Tolerance (PBFT) or Delegated Proof of Stake (DPoS) for efficient agreement on model updates.

Benefits:

Reduces latency compared to Proof of Work (PoW).

Ensures high fault tolerance and security.

Aggregation Process:

Use Homomorphic Encryption to aggregate updates directly in their encrypted form.

Smart contracts handle the aggregation process, ensuring compliance and consistency.

Example aggregation algorithm: Federated Averaging (FedAvg) or weighted aggregation.

Tools:

Aggregation Script: Implement using Python or Solidity smart contracts.

4.2.4 Global Model Distribution

The aggregated global model is encrypted and distributed back to participating nodes.

Nodes decrypt the global model and use it to improve local training.

Optimization: Use asynchronous FL techniques to allow staggered updates, improving overall efficiency.

4.3 Tools and Technologies

4.3.1 Blockchain Platforms

Hyperledger Fabric:

Permissioned blockchain ideal for managing sensitive data.

Features: Identity management, channel-based communication.

Ethereum/Polygon:

Decentralized public blockchain for transparency and audit trails.

4.3.2 Federated Learning Frameworks

PySyft: A privacy-preserving FL library compatible with PyTorch.

TensorFlow Federated (TFF): Supports cross-device and cross-silo federated learning.

4.3.3 Encryption Libraries

TenSEAL: A Python library for Homomorphic Encryption tailored for deep learning.

PyCryptodome: For cryptographic operations like key generation and encryption.

4.3.4 Programming Frameworks

Web3.py: Facilitates blockchain interactions, including smart contract deployment and transaction submission.

SmartPy: For developing smart contracts efficiently.

Python or Solidity: For scripting and implementing aggregation logic.

4.4 Deployment Plan

4.4.1 Testing Environment

Simulate a multi-node network using Docker containers or Kubernetes clusters to represent distributed nodes.

Use Ganache or Hyperledger Fabric for local blockchain testing.

4.4.2 Deployment Stages

Proof of Concept:

Test on small-scale datasets (e.g., Kaggle datasets).

Validate end-to-end workflows, including FL, Blockchain, and HE interactions.

Pilot Deployment: Implement the framework in a controlled real-world environment, such as a consortium of financial institutions or hospitals.

Full-Scale Deployment: Roll out across multiple industries, using scalable blockchain platforms like Polygon.

4.4.3 Monitoring and Maintenance

Use blockchain monitoring tools like Etherscan for public networks.

Implement logging and monitoring scripts for federated updates and node reliability tracking.

4.5 Key Benefits of the Implementation Plan

Scalability: Sharded blockchain and Layer 2 solutions enable large-scale deployments without compromising performance.

Privacy and Security: HE, Differential Privacy, and ZKP provide robust privacy guarantees. Blockchain's immutability ensures trust and transparency.

Efficiency: Edge computing and compressed updates reduce computational and communication overhead.

Broad Applicability: The architecture supports diverse domains, from healthcare to IoT, enabling real-world impact.

6. Conclusion

By addressing scalability, computational efficiency, and domain adaptability, the proposed enhancements elevate Blockchain-FL-HE frameworks to a new level of practicality and impact. The integration of advanced cryptographic techniques, real-world validation, and cross-domain applications ensures the framework remains relevant in evolving technological landscapes. These improvements will enable scalable, efficient, and secure decentralized learning systems with broad applicability in finance, healthcare, IoT, and beyond.

7. References

- [1] D. Zhang, Q. Yang, and L. Chen, "Federated learning with blockchain for autonomous driving," *IEEE Intelligent Systems*, vol. 34, no. 5, pp. 35–45, 2019.
- [2] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.
- [3] H. B. McMahan, E. Moore, D. Ramage, and B. Hampson, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, Ft. Lauderdale, FL, USA, 2017, pp. 1273–1282.
- [4] A. Sharma and S. Bhattacharya, "Blockchain-based federated learning for secure data aggregation," *IEEE Internet of Things Journal*, vol. 7, no. 6, pp. 5018–5028, 2020. Available: [IEEE IoT Journal](#).
- [5] M. Gupta et al., "Blockchain-based Federated Learning for Securing Internet of Things: A Comprehensive Survey," 2022. Available: [ResearchGate](#).
- [6] T. Pham, H. Nguyen, and T. Hoang, "Federated learning with blockchain: A privacy-preserving architecture for medical applications," *Journal of Healthcare Engineering*, vol. 2021, Article ID 8578663, 2021. Available: [PubMed Central](#).
- [7] Y. Qu, M. P. Uddin, C. Gan, Y. Xiang, and L. Gao, "Blockchain-enabled federated learning: A survey," *ACM Computing Surveys*, 2022. Available: [ACM Computing Surveys](#).
- [8] M. Al Asqah and T. Moulahi, "Federated learning and blockchain integration for privacy protection in the internet of things: Challenges and solutions," *Future Internet*, vol. 15, no. 6, p. 203, 2023. Available: [MDPI](#).
- [9] Y. Lu, X. Huang, Y. Dai, S. Maharjan, and Y. Zhang, "Blockchain and federated learning for privacy-preserved data sharing in industrial IoT," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 6, pp. 4177–4186, 2019. Available: [IEEE Explore](#).
- [10] Y. Lu, X. Huang, K. Zhang, S. Maharjan, and Y. Zhang, "Blockchain and federated learning for 5G beyond," *IEEE Network*, vol. 34, no. 6, pp. 6–12, 2020. Available: [IEEE Network](#).
- [11] Y. Li, C. Chen, N. Liu, H. Huang, Z. Zheng, and Q. Yan, "A blockchain-based decentralized federated learning framework with committee consensus," *IEEE Network*, vol. 34, no. 4, pp. 78–85, 2020. Available: [IEEE Explore](#).
- [12] JPMorgan, "Federated Learning Meets Blockchain," Available: [JPMorgan](#).
- [13] S. Vivek, "Blockchain Technology and Its Integration with Federated Learning: A Comprehensive Survey," *School of Information Technology, Deakin University*, Melbourne, Victoria, Australia.

- [14] Suni639, "fraudsters_phase3a," GitHub repository, Available: GitHub.
- [15] Kaggle, "Credit Card Fraud Detection," Kaggle dataset, Available: Kaggle.

Acknowledgment

This report builds upon prior work completed by Shreyas Vivek from the School of Information Technology, Deakin University, Melbourne, Victoria, Australia. I acknowledge his foundational contributions to the integration of Blockchain, FL, and HE, which have guided this extended analysis and enhancement.

I also recognize the use of OpenAI's ChatGPT in assisting with drafting, organizing, and refining this report. The insights generated by ChatGPT have been critically reviewed and integrated into this report to complement and extend Shreyas Vivek's initial work.

For further inquiries, please contact: Shreyas Vivek
School of Information Technology
Deakin University, Melbourne, Victoria, Australia
Email: vivekshreyas01@gmail.com